

Практическое занятие 23: Создание моделей и миграций в Django

Цель занятия:

- Научиться создавать модели для хранения информации о мероприятиях, категориях и пользователях.
 - Понять процесс создания и применения миграций в Django.
-

Задачи занятия

1. Введение в модели и миграции Django
 2. Создание модели `Category`
 3. Создание модели `Event`
 4. Создание модели `Registration`
 5. Создание модели `Review`
 6. Создание модели `Comment`
 7. Создание модели `Profile`
 8. Создание и применение миграций
 9. Запуск и проверка приложения
 10. Выполнение практического упражнения. Создание собственной модели
 11. Выполнение дополнительных заданий. Добавление моделей и методов
-

Задача 1: Введение в модели и миграции Django

1.1. Что такое модели в Django?

- **Модель** — это класс в Django, который представляет таблицу в базе данных.
- Модели определяют структуру данных, включая поля и типы данных.
- Позволяют взаимодействовать с базой данных с помощью объектов Python, используя ORM (Object-Relational Mapping).

1.2. Что такое миграции в Django?

- **Миграции** — это механизм Django для управления изменениями в модели данных.

- Они позволяют синхронизировать модели с базой данных.
 - Команды `makemigrations` и `migrate` используются для создания и применения миграций.
-

Задача 2: Создание модели `Category`

2.1. Открытие файла `models.py`

- Откройте файл `events/models.py` в вашем редакторе кода.

2.2. Импорт необходимых модулей

```
# events/models.py

from django.db import models
```

2.3. Создание класса `Category`

```
# events/models.py

class Category(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField(blank=True)

    def __str__(self):
        return self.name
```

Пояснение:

- `name` : Название категории, ограниченное 100 символами.
- `description` : Описание категории, может быть пустым (`blank=True`).
- **Метод `__str__`** : Определяет строковое представление объекта, полезно в админ-панели.

2.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 3: Создание модели `Event`

3.1. Добавление класса `Event` в `models.py`

```
# events/models.py

class Event(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    location = models.CharField(max_length=200)
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='events')

    def __str__(self):
        return self.title
```

Пояснение:

- `title` : Название мероприятия.
- `description` : Описание мероприятия.
- `date` : Дата и время проведения.
- `location` : Место проведения.
- `category` : Связь с моделью `Category` , при удалении категории удаляются связанные мероприятия (`on_delete=models.CASCADE`).
- `related_name='events'` : Позволяет обращаться к мероприятиям из категории через `category.events.all()` .

3.2. Сохраните изменения

- Сохраните файл `models.py` .

Задача 4: Создание модели `Registration`

4.1. Импорт модели `User`

```
# events/models.py

from django.contrib.auth.models import User
```

4.2. Добавление класса `Registration`

```
# events/models.py
```

```
class Registration(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='registrations')
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='registrations')
    registration_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.user.username} зарегистрировался на {self.event.title}'
```

Пояснение:

- `user`: Связь с моделью `User`, представляющей пользователей.
- `event`: Связь с моделью `Event`.
- `registration_date`: Дата и время регистрации, автоматически устанавливается при создании (`auto_now_add=True`).
- `related_name='registrations'`: Позволяет обращаться к регистрациям пользователя или мероприятия.

4.3. Сохраните изменения

- Сохраните файл `models.py`.

Задача 5: Создание модели `Review`

5.1. Добавление класса `Review`

```
# events/models.py

class Review(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='reviews')
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='reviews')
    rating = models.IntegerField()
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Отзыв {self.user.username} о {self.event.title}'
```

Пояснение:

- `rating` : Оценка мероприятия (например, от 1 до 5).
- `comment` : Текст отзыва.
- `created_at` : Дата и время создания отзыва.

5.2. Сохраните изменения

- Сохраните файл `models.py`.
-

Задача 6: Создание модели `Comment`

6.1. Добавление класса `Comment`

```
# events/models.py

class Comment(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='comments')
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='comments')
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Комментарий {self.user.username} к {self.event.title}'
```

Пояснение:

- `content` : Содержимое комментария.
- `created_at` : Дата и время создания комментария.

6.2. Сохраните изменения

- Сохраните файл `models.py`.
-

Задача 7: Создание модели `Profile`

7.1. Добавление класса `Profile`

```
# events/models.py

from django.db.models.signals import post_save
```

```

from django.dispatch import receiver

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)
    subscribed = models.BooleanField(default=False)

    def __str__(self):
        return f'Профиль {self.user.username}'

```

Пояснение:

- `OneToOneField`: Связь "один к одному" с моделью `User`.
- `bio`: Биография пользователя.
- `avatar`: Аватар пользователя, сохраняется в директорию `avatars/`.
- `subscribed`: Флаг подписки на рассылку.

7.2. Создание сигналов для автоматического создания и сохранения профиля

```

# events/models.py

@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

@receiver(post_save, sender=User)
def save_user_profile(sender, instance, **kwargs):
    instance.profile.save()

print("Модели успешно созданы")

```

Пояснение:

- **Сигналы** `post_save`: Автоматически создают профиль при создании нового пользователя и сохраняют изменения в профиле при сохранении пользователя.

7.3. Сохраните изменения

- Сохраните файл `models.py`.

Задача 8: Создание и применение миграций

8.1. Создание миграций

- Откройте терминал в корневой директории вашего проекта.

```
python manage.py makemigrations
```

Пояснение:

- Команда `makemigrations` создаст файлы миграций на основе изменений в моделях.

8.2. Применение миграций

```
python manage.py migrate
```

Пояснение:

- Команда `migrate` применит миграции к базе данных, создавая необходимые таблицы.

Задача 9: Запуск и проверка приложения

9.1. Запуск сервера разработки

```
python manage.py runserver
```

9.2. Проверка

- Перейдите по адресу <http://127.0.0.1:8000/> и убедитесь, что сайт работает без ошибок.
В терминале выведется сообщение `Модели успешно созданы`.

9.3. Проверка базы данных

- Вы можете использовать админ-панель Django для просмотра и редактирования данных.
- Создайте суперпользователя:

```
python manage.py createsuperuser
```

- Следуйте инструкциям для создания учетной записи администратора.

- Перейдите по адресу <http://127.0.0.1:8000/admin/> и войдите под учетной записью администратора.

Задача 10: Практическое упражнение

Задание для студентов:

- Создайте новую модель `Sponsor` в `events/models.py`, которая будет хранить информацию о спонсорах мероприятий.
- Поля модели могут включать:
 - `name` (`CharField`)
 - `logo` (`ImageField`)
 - `website` (`URLField`)
 - `events` (`ManyToManyField` с моделью `Event`)
- Примените миграции и убедитесь, что модель создана в базе данных.

Решение:

1. Добавление класса `Sponsor`:

```
# events/models.py

class Sponsor(models.Model):
    name = models.CharField(max_length=200)
    logo = models.ImageField(upload_to='sponsors/', blank=True, null=True)
    website = models.URLField(blank=True)
    events = models.ManyToManyField(Event, related_name='sponsors',
    blank=True)

    def __str__(self):
        return self.name
```

2. Применение миграций:

```
python manage.py makemigrations
python manage.py migrate
```

Дополнительное задание

1. Добавление модели `Venue`

Цель: Создать модель `Venue` для хранения информации о местах проведения мероприятий и связать её с моделью `Event`.

Шаги:

1. Создайте модель `Venue` в `events/models.py`:

```
class Venue(models.Model):
    name = models.CharField(max_length=200)
    address = models.CharField(max_length=300)
    capacity = models.IntegerField()

    def __str__(self):
        return self.name
```

2. Обновите модель `Event` для связи с `Venue`:

```
class Event(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='events')
    venue = models.ForeignKey(Venue, on_delete=models.CASCADE,
related_name='events')

    def __str__(self):
        return self.title
```

3. Примените миграции:

```
python manage.py makemigrations
python manage.py migrate
```

2. Добавление метода `get_absolute_url` в модель `Event`

Цель: Добавить метод для получения URL мероприятия, что упростит навигацию и ссылку на объект.

Шаги:

1. Импортируйте функцию `reverse`:

```
from django.urls import reverse
```

2. Добавьте метод `get_absolute_url` в класс `Event`:

```
class Event(models.Model):
    # ... ваши поля ...

    def get_absolute_url(self):
        return reverse('events:event_detail', args=[str(self.id)])
```

Пояснение:

- Теперь вы можете использовать `event.get_absolute_url()` для получения URL мероприятия.

3. Настройка отображения связей в админ-панели

Цель: Улучшить админ-панель, чтобы можно было просматривать связанные объекты.

Шаги:

1. Откройте файл `events/admin.py`.

2. Зарегистрируйте модели и настройте отображение:

```
# events/admin.py

from django.contrib import admin
from .models import Category, Event, Registration, Review, Comment, Profile,
Venue, Sponsor

@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('name', 'description')
    search_fields = ('name',)

@admin.register(Event)
class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'category', 'venue')
    list_filter = ('category', 'date')
    search_fields = ('title', 'description')
    date_hierarchy = 'date'

@admin.register(Venue)
class VenueAdmin(admin.ModelAdmin):
    list_display = ('name', 'address', 'capacity')
    search_fields = ('name', 'address')

# Зарегистрируйте остальные модели
admin.site.register(Registration)
admin.site.register(Review)
admin.site.register(Comment)
```

```
admin.site.register(Profile)
admin.site.register(Sponsor)
```

Пояснение:

- Использование декораторов `@admin.register` для регистрации моделей.
 - Настройка отображения списка объектов и полей для поиска.
-

Заключение

Поздравляем! Вы успешно:

- Создали модели для хранения информации о категориях, мероприятиях, пользователях и других сущностях.
 - Научились создавать и применять миграции в Django.
 - Настроили админ-панель для управления данными.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Что такое миграции и почему они важны?**
 - Миграции позволяют отслеживать изменения в моделях и применять их к базе данных, обеспечивая синхронизацию структуры данных.
 - **Как устанавливаются связи между моделями в Django?**
 - С помощью полей `ForeignKey`, `OneToOneField`, `ManyToManyField` и аргументов `related_name`, `on_delete`.
 - **Как сигналы помогают в работе с моделями?**
 - Сигналы позволяют выполнять действия при определенных событиях, например, автоматически создавать профиль при создании пользователя.
-

Полезные ресурсы

- [Документация Django: Модели](#)
- [Документация Django: Миграции](#)
- [Документация Django: Админ-панель](#)
- [Django ORM Cookbook](#)